Proposta de Projeto
ES47C/IF66K - Oficina de Integração 2
Prof. Antonio Carlos Fernandes da Silva

1 Descrição do projeto

A disciplina de Oficina de Integração 2 tem como objetivo avaliar a capacidade do aluno em integrar conteúdos de disciplinas anteriores no curso. Dada a organização do curso de Engenharia de Software, a disciplina de **Oficina de Integração 2** terá como foco a integração de conteúdos das seguintes disciplinas:

• Programação Orientada a Objetos (1 e 2);
• Banco de Dados;
• Programação Web (*Front-End* e *Back-End*);
• Teste de Software;
• Gerência de Configuração e Manutenção de Software;
• Arquitetura de Software
• Gerenciamento de Projeto de Software.

A disciplina de Oficina de Integração 2, é caracterizada como extensionista, tendo sua carga horária contabilizada como horas de extensão para os alunos. Nesse semestre, os alunos devem elaborar projetos de sistemas para o projeto de extensão **ELLP - Ensino Lúdico de Lógica e Programação** [1].
Cada projeto deve focar em UMA das propostas descritas abaixo.

- Ferramenta de programação em blocos que gere código em portugol e/ou linguagem C, com os movimentos básicos de um robô (para frente, para trás, direita, esquerda, repetir e condicional Se).

- Ferramenta para tradução de código em linguagem de programação do arduino para programação em blocos, permitindo a visualização dos blocos relativos ao código.

- Ferramenta visual para programação de um pequeno personagem que permita andar pela tela usando comandos básicos (frente, trás, direita, esquerda). O controle pode ser feito com botões na tela ou setas do teclado.

- Ferramenta visual para programação de um pequeno personagem que permita andar pela tela usando comandos básicos (frente, trás, direita, esquerda). O controle pode ser feito por código em portugol.

Todas as aplicações devem permitir cadastro dos usuários e login com o google.

As equipes poderão ser organizadas com até 5 alunos. A avaliação será individual, considerando a contribuição, em termos de autoria de código-fonte, de cada membro da equipe ao projeto.

O projeto será desenvolvido ao longo do semestre e será dividido em duas fases principais: planejamento e implementação. A fase de implementação, por sua vez, será subdividida em múltiplas iterações de desenvolvimento.

---

1 https://www.utfpr.edu.br/campus/cornelioprocopio/extensao/atividades-extensao/projeto-de-extensao-ellp-ensino-ludico-de-logica-e-programacao

2 Fases do projeto

2.1 Planejamento

Nessa fase, os alunos deverão se organizar em grupos e definir os temas de seus projetos. Os seguintes objetivos devem ser alcançados nessa etapa:

Definição de requisitos funcionais;
Definição da arquitetura em alto nível do sistema;
Definição da estratégia de automação de testes do sistema;
Definição das tecnologias a serem utilizadas no projeto.
Definição de cronograma

Ao final da fase do planejamento, os alunos devem apresentar um repositório com artefatos iniciais do projeto **(README, arquivos de configuração do ambiente, definição dos testes automatizados e diagrama com a arquitetura do projeto)** e um repositório dos requisitos funcionais a serem desenvolvidos.

2.1.1. Reunião obrigatória - Agendada no mínimo uma a cada 2 semanas, por grupo

Os requisitos funcionais e a arquitetura do sistema deverão ser discutidos com o professor, para delimitar o escopo do projeto. As tecnologias a serem utilizadas também devem ser previamente analisadas pelo professor, para verificar a viabilidade de uso das mesmas, bem como o conhecimento prévio do professor.

2.2 Implementação

O projeto deverá utilizar o **Scrum** como metodologia de desenvolvimento. Mas considerando a carga horária da disciplina, serão realizados dois *sprints*. O *backlog* deve considerar as funcionalidades definidas na fase de planejamento.
A avaliação do projeto será realizada na etapa de *sprint review*, ao final dos *sprints*.
Deverá ser utilizada a plataforma **Github (aberto)** como repositório de código-fonte do projeto.
As funcionalidades a serem desenvolvidas em cada *sprint* deverão ser documentadas como *issues* no repositório do projeto e deverão ser resolvidas com *commits* na plataforma Github.
O gerenciamento das tarefas deve ser realizado utilizando um *Kanban*. Para todas as funcionalidades, deverão ser elaborados scripts de teste automatizados, utilizando estratégias de teste apropriadas, dependendo da plataforma tecnológica utilizada pelo grupo. A implementação das funcionalidades também deverá ser realizada considerando a arquitetura do projeto definida na fase de planejamento.
Ao final dos sprints, os alunos serão avaliados considerando:

- participação no projeto, considerando suas contribuições individuais no repositório;
- elaboração de casos de teste suficientes para as funcionalidades (será utilizada a métrica de cobertura, para também avaliar os casos de teste gerados);
- conformidade com o processo de desenvolvimento, utilizando os *issues* do *Github* para documentar o progresso do projeto e ferramenta de Kanban;
- implementação das funcionalidades definidas para cada *sprint*;
- adequação da implementação das funcionalidades com a arquitetura definida no projeto;
- ajustes das implementações, de acordo com revisões e comentários realizados pelo professor.

Ao final de cada *sprint*, deverá ser enviado um vídeo de no **máximo 5 minutos**, apresentando a execução do projeto, execução dos testes automatizados e as funcionalidades implementadas no *sprint*.
Após uma avaliação inicial do professor, considerando o vídeo e os artefatos no *Github*, serão definidos ajustes necessários ao projeto pelo professor (*bugs*, ajustes ou novos requisitos associados). Essa etapa, ao final dos *sprints*, contará como abordagem de recuperação de nota, que os alunos poderão implementar em seus projetos, para recuperar notas perdidas ao longo do projeto. A etapa final de revisão/recuperação dos *sprints* deve durar 1 semana.